Lab Assignment-01 || Roll no. 23103163

Title: Image Loading & Basic Operations in Python (Blur, Smooth, Edges, Contours)

Task 1: WAP to Read, Display, Save Copy of image

```
import cv2

import numpy as np

import os

BASE_DIR = os.getcwd()

image_path = os.path.join(BASE_DIR, "vik.png")

print("Looking for image at:", image_path)

img = cv2.imread(image_path)

if img is None:

    print("Error: Image not found or cannot be read.")

else:

    height, width = img.shape[:2]

    channels = 1 if len(img.shape) == 2 else img.shape[2]

    print(f"Width: {width}")

    print(f"Height: {height}")

    print(f"Channels: {channels}")

    output_dir = os.path.join(BASE_DIR, "outputs")

    os.makedirs(output_dir, exist_ok=True)

    cv2.imwrite(os.path.join(output_dir, "copy.png"), img)
img_rgb = cv2.cvtColor(img_to_display, cv2.COLOR_BGR2RGB)

    # Display the image

    plt.figure(figsize=(10, 10))

    plt.imshow(img_rgb)

    plt.title("Displayed Image: outputs/copy.png")

    plt.axis('off')

    plt.show()

    print("Image saved as outputs/copy.png")
```

output:-

```
Looking for image at: /content/aabb.png
Width: 1536
Height: 2752
Channels: 3
Image saved as outputs/copy.png
```

Displayed Image: outputs/copy.png

Task 2- Convert to Grayscale +Inspect Pixels

```
import cv2

import matplotlib.pyplot as plt

import os

# Get the base directory

BASE_DIR = os.getcwd()

output_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")
```

```
img_color = cv2.imread(output_image_path)
if img_color is None:
    print(f"Error: Could not read image at {output_image_path}")
else:
    gray_img = cv2.cvtColor(img_color, cv2.COLOR_BGR2GRAY)
    plt.figure(figsize=(10, 10))
    plt.imshow(gray_img, cmap='gray')
    plt.title("Grayscale Image")
    plt.axis('off')
    plt.show()
    output_dir = os.path.join(BASE_DIR, "outputs")
    os.makedirs(output_dir, exist_ok=True)
    gray_image_path = os.path.join(output_dir, "gray.png")
    cv2.imwrite(gray_image_path, gray_img)
    print(f"Grayscale image saved as {gray_image_path}")
    # Print grayscale image shape
    print(f"Grayscale image shape (H, W): {gray_img.shape}")
    if gray_img.shape[0] > 50 and gray_img.shape[1] > 50:
        sample_pixel_value = gray_img[50, 50]
        print(f"Value of pixel at [50, 50]: {sample_pixel_value}")
    else:
        print("Image is too small to sample pixel at [50,50]")
```

Grayscale Image

Task 3- Basic operation Resize ,crop, rotate, flip

Resize:-

```
import cv2

import os

import matplotlib.pyplot as plt

# Get the base directory

BASE_DIR = os.getcwd()

input_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")

output_resize_half_path = os.path.join(BASE_DIR, "outputs", "resize_half.png")
```

```
# Read the image
img = cv2.imread(input_image_path)
if img is None:
    print(f"Error: Could not read image at {input_image_path}")
else:
    # Calculate new dimensions (50% of original)
    height, width = img.shape[:2]
    new_width = int(width * 0.5)
    new_height = int(height * 0.5)
    resized_img = cv2.resize(img, (new_width, new_height), interpolation=cv2.INTER_AREA)
    os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
    cv2.imwrite(output_resize_half_path, resized_img)
    print(f"Resized image saved as {output_resize_half_path}")
    plt.figure(figsize=(8, 8))
    plt.imshow(cv2.cvtColor(resized_img, cv2.COLOR_BGR2RGB))
    plt.title("Resized Image (50%)")
    plt.axis('off')
    plt.show()
    print(f"Original shape: {img.shape}")
    print(f"Resized shape: {resized_img.shape}")
```

output:-

```
Resized image saved as /content/outputs/resize_half.png
```

Resized Image (50%)

Crop:-

```
import cv2
import os
import matplotlib.pyplot as plt
# Get the base directory
BASE_DIR = os.getcwd()
input_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")
output_crop_path = os.path.join(BASE_DIR, "outputs", "crop.png")
# Read the image
img = cv2.imread(input_image_path)
if img is None:
    print(f"Error: Could not read image at {input_image_path}")
```

```
else:
    height, width = img.shape[:2]
    # Calculate center coordinates
    center_x = width // 2
    center_y = height // 2
    # Define crop size
    crop_size = 200
    half_crop_size = crop_size // 2
    start_x = max(0, center_x - half_crop_size)
    end_x = min(width, center_x + half_crop_size)
    start_y = max(0, center_y - half_crop_size)
    end_y = min(height, center_y + half_crop_size)
    # Perform crop
    cropped_img = img[start_y:end_y, start_x:end_x]
    # Save the cropped image
    os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
    cv2.imwrite(output_crop_path, cropped_img)
    print(f"Cropped image saved as {output_crop_path}")
    plt.figure(figsize=(5, 5))
    plt.imshow(cv2.cvtColor(cropped_img, cv2.COLOR_BGR2RGB))
    plt.title(f"Cropped Image ({crop_size}x{crop_size} from center)")
    plt.axis('off')
    plt.show()

    print(f"Cropped image shape: {cropped_img.shape}")
```

```
Cropped image saved as /content/outputs/crop.png
```

Cropped Image (200x200 from center)

```
Cropped image shape: (200, 200, 3)
```

Flip:-

```
import cv2
import os
import matplotlib.pyplot as plt
# Get the base directory
BASE_DIR = os.getcwd()
input_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")
output_flip_h_path = os.path.join(BASE_DIR, "outputs", "flip_h.png")
# Read the image
img = cv2.imread(input_image_path)
if img is None:
    print(f"Error: Could not read image at {input_image_path}")
else:
    # Perform horizontal flip (flip code 1)
    flipped_h_img = cv2.flip(img, 1)
    os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
```

```
cv2.imwrite(output_flip_h_path, flipped_h_img)
print(f"Horizontally flipped image saved as {output_flip_h_path}")
plt.figure(figsize=(10, 10))
plt.imshow(cv2.cvtColor(flipped_h_img, cv2.COLOR_BGR2RGB))
plt.title("Horizontally Flipped Image")
plt.axis('off')
plt.show()
```

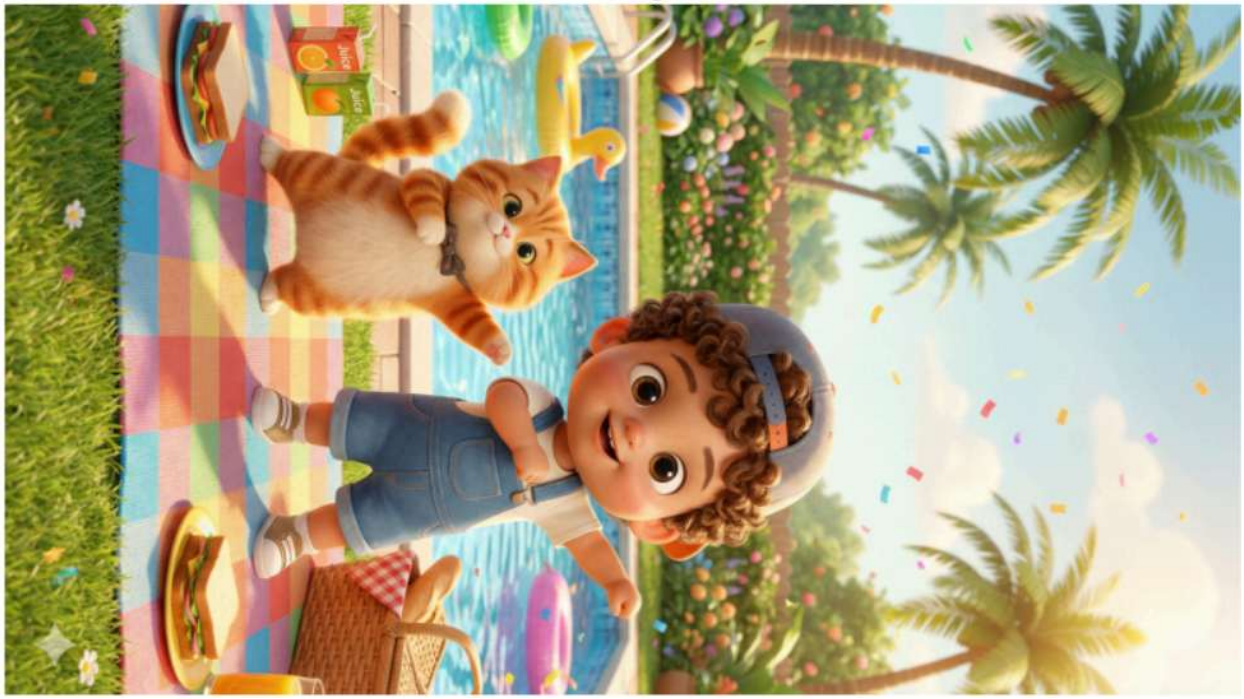
Rotate:-

```
import cv2
import os
import matplotlib.pyplot as plt
```

```
# Get the base directory
BASE_DIR = os.getcwd()
input_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")
output_rotate_90_path = os.path.join(BASE_DIR, "outputs", "rotate_90.png")
# Read the image
img = cv2.imread(input_image_path)
if img is None:
    print(f"Error: Could not read image at {input_image_path}")
else:
    rotated_90_img = cv2.rotate(img, cv2.ROTATE_90_CLOCKWISE)
    # Save the rotated image
    os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
    cv2.imwrite(output_rotate_90_path, rotated_90_img)
    print(f"Image rotated by 90 degrees and saved as {output_rotate_90_path}")
    plt.figure(figsize=(10, 10))
    plt.imshow(cv2.cvtColor(rotated_90_img, cv2.COLOR_BGR2RGB))
    plt.title("Rotated Image (90°)")
    plt.axis('off')
    plt.show()
```

```
Image rotated by 90 degrees and saved as /content/outputs/rotate_90.png
```

Rotated Image (90°)



Task 4: Brightness,Contrast,Negative

Brightness:-

```
import cv2
import os
import matplotlib.pyplot as plt
import numpy as np
# Get the base directory
BASE_DIR = os.getcwd()
input_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")
output_bright_plus_path = os.path.join(BASE_DIR, "outputs", "bright_plus.png")
output_bright_minus_path = os.path.join(BASE_DIR, "outputs", "bright_minus.png")
# Read the image
img = cv2.imread(input_image_path)
if img is None:
    print(f"Error: Could not read image at {input_image_path}")
else:
    beta_plus = 50
    beta_minus = -50
```

```
# Increase brightness
bright_plus_img = cv2.add(img, beta_plus)
# Decrease brightness
bright_minus_img = cv2.subtract(img, abs(beta_minus))
# Save the images
os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
cv2.imwrite(output_bright_plus_path, bright_plus_img)
print(f"Increased brightness image saved as {output_bright_plus_path}")
cv2.imwrite(output_bright_minus_path, bright_minus_img)
print(f"Decreased brightness image saved as {output_bright_minus_path}")
# Display the images
plt.figure(figsize=(15, 7))
plt.subplot(1, 2, 1)
plt.imshow(cv2.cvtColor(bright_plus_img, cv2.COLOR_BGR2RGB))
plt.title("Increased Brightness")
plt.axis('off')
plt.subplot(1, 2, 2)
plt.imshow(cv2.cvtColor(bright_minus_img, cv2.COLOR_BGR2RGB))
plt.title("Decreased Brightness")
plt.axis('off')
plt.show()
```

```
Increased brightness image saved as /content/outputs/bright_plus.png
Decreased brightness image saved as /content/outputs/bright_minus.png
```

Increased Brightness

Decreased Brightness

Contrast:-

```
import cv2
import os
import matplotlib.pyplot as plt
# Get the base directory
BASE_DIR = os.getcwd()
input_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")
output_contrast_high_path = os.path.join(BASE_DIR, "outputs", "contrast_high.png")
# Read the image
img = cv2.imread(input_image_path)
if img is None:
    print(f"Error: Could not read image at {input_image_path}")
else:
    alpha = 1.5
    beta = 0
    contrast_high_img = cv2.convertScaleAbs(img, alpha=alpha, beta=beta)
    # Save the image
```

```
os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
cv2.imwrite(output_contrast_high_path, contrast_high_img)
print(f"Increased contrast image saved as {output_contrast_high_path}")
# Display the image
plt.figure(figsize=(10, 10))
plt.imshow(cv2.cvtColor(contrast_high_img, cv2.COLOR_BGR2RGB))
plt.title("Increased Contrast")
plt.axis('off')
plt.show()
```

```
Increased contrast image saved as /content/outputs/contrast_high.png
```

Negative Image inversion:-

```
import cv2
import os
import matplotlib.pyplot as plt
# Get the base directory
BASE_DIR = os.getcwd()
input_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")
output_negative_path = os.path.join(BASE_DIR, "outputs", "negative.png")
# Read the image
img = cv2.imread(input_image_path)
if img is None:
    print(f"Error: Could not read image at {input_image_path}")
else:
    # Invert pixel values (255 - pixel_value)
    negative_img = cv2.bitwise_not(img)
    # Save the image
    os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
    cv2.imwrite(output_negative_path, negative_img)
    print(f"Negative image saved as {output_negative_path}")
    # Display the image
    plt.figure(figsize=(10, 10))
    plt.imshow(cv2.cvtColor(negative_img, cv2.COLOR_BGR2RGB))
    plt.title("Negative Image (Inversion)")
    plt.axis('off')
    plt.show()
```

Negative Image (Inversion)

Task 5:Thresholding (Required for Contours)

```
import cv2

import os

import matplotlib.pyplot as plt

# Get the base directory

BASE_DIR = os.getcwd()

input_gray_image_path = os.path.join(BASE_DIR, "outputs", "gray.png")

output_thresh_simple_path = os.path.join(BASE_DIR, "outputs", "thresh_simple.png")

# Read the grayscale image
```

```
gray_img = cv2.imread(input_gray_image_path, cv2.IMREAD_GRAYSCALE)
if gray_img is None:
    print(f"Error: Could not read grayscale image at {input_gray_image_path}")
else:
    # Define threshold value
    threshold_value = 127
    _, thresh_img = cv2.threshold(gray_img, threshold_value, 255, cv2.THRESH_BINARY)
    # Save the thresholded image
    os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
    cv2.imwrite(output_thresh_simple_path, thresh_img)
    print(f"Thresholded image saved as {output_thresh_simple_path}")
    # Display the thresholded image
    plt.figure(figsize=(10, 10))
    plt.imshow(thresh_img, cmap='gray')
    plt.title(f"Binary Image (Threshold = {threshold_value})")
    plt.axis('off')
    plt.show()
    print(f"Thresholded image shape: {thresh_img.shape}")
    # Check a sample pixel value for observation
    if thresh_img.shape[0] > 50 and thresh_img.shape[1] > 50:
        sample_pixel_value_original = gray_img[50, 50]
        sample_pixel_value_thresh = thresh_img[50, 50]
        print(f"Original grayscale pixel at [50, 50]: {sample_pixel_value_original}")
        print(f"Thresholded pixel at [50, 50]: {sample_pixel_value_thresh}")
```

Task6-Reduce noise and make contours/edges better.

```
import cv2
import os
import matplotlib.pyplot as plt
# Get the base directory
BASE_DIR = os.getcwd()
input_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")
output_blur_avg_path = os.path.join(BASE_DIR, "outputs", "blur_avg.png")
# Read the image
```

```
img = cv2.imread(input_image_path
if img is None:
    print(f"Error: Could not read image at {input_image_path}")
else:
    # Apply Averaging Blur
    blur_avg_img = cv2.blur(img, (5, 5))
    # Save the blurred image
    os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
    cv2.imwrite(output_blur_avg_path, blur_avg_img)
    print(f"Averaging blurred image saved as {output_blur_avg_path}")
    # Display the blurred image (convert BGR to RGB for matplotlib)
    plt.figure(figsize=(10, 10))
    plt.imshow(cv2.cvtColor(blur_avg_img, cv2.COLOR_BGR2RGB))
    plt.title("Averaging Blur")
    plt.axis('off')
    plt.show()
```

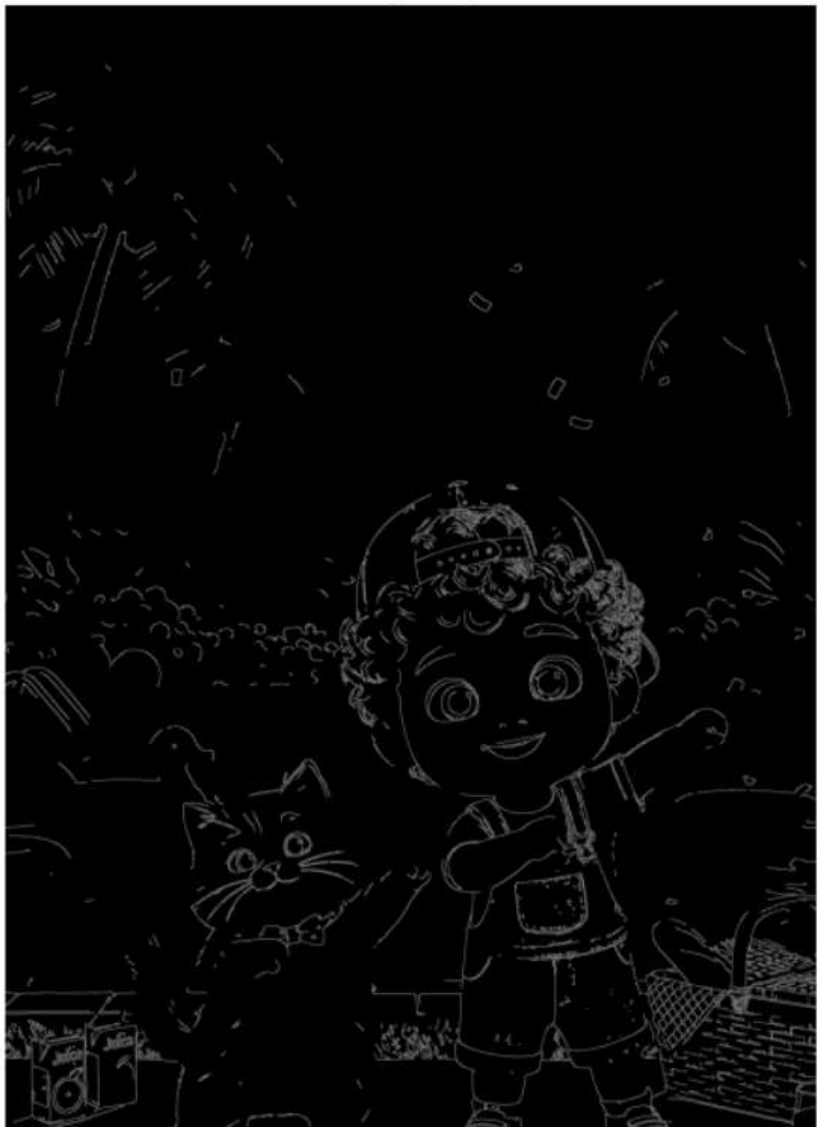
Averaging Blur

Task 7:- Detect boundaries of objects.

```
import cv2

import os

import matplotlib.pyplot as plt

# Get the base directory

BASE_DIR = os.getcwd()

input_gray_image_path = os.path.join(BASE_DIR, "outputs", "gray.png") # Using the previously saved grayscale image

output_edges_path = os.path.join(BASE_DIR, "outputs", "edges.png")
```

```
# Read the grayscale image
gray_img = cv2.imread(input_gray_image_path, cv2.IMREAD_GRAYSCALE)
if gray_img is None:
    print(f"Error: Could not read grayscale image at {input_gray_image_path}")
else:
    edges_img = cv2.Canny(gray_img, threshold1=100, threshold2=200)
    # Save the edges image
    os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
    cv2.imwrite(output_edges_path, edges_img)
    print(f"Edges image saved as {output_edges_path}")
    # Display the edges image
    plt.figure(figsize=(10, 10))
    plt.imshow(edges_img, cmap='gray')
    plt.title("Canny Edges")
    plt.axis('off')
    plt.show()
```

Canny Edges



Task8-Find object shapes and draw boundaries.

```
import cv2

import os

import matplotlib.pyplot as plt

# Get the base directory

BASE_DIR = os.getcwd()

input_original_image_path = os.path.join(BASE_DIR, "outputs", "copy.png")

input_gray_image_path = os.path.join(BASE_DIR, "outputs", "gray.png")

output_binary_path = os.path.join(BASE_DIR, "outputs", "binary_for_contours.png")

output_contours_path = os.path.join(BASE_DIR, "outputs", "contours.png")

# Read the original image
```

```
img_original = cv2.imread(input_original_image_path)
# Read the grayscale image
gray_img = cv2.imread(input_gray_image_path, cv2.IMREAD_GRAYSCALE)
if img_original is None:
    print(f"Error: Could not read original image at {input_original_image_path}")
elif gray_img is None:
    print(f"Error: Could not read grayscale image at {input_gray_image_path}")
else:
    # 1. Blur the grayscale image to reduce noise
    blurred_img = cv2.GaussianBlur(gray_img, (5, 5), 0)
    _, binary_img = cv2.threshold(blurred_img, 127, 255, cv2.THRESH_BINARY)
    # Save the binary image for contours
    os.makedirs(os.path.join(BASE_DIR, "outputs"), exist_ok=True)
    cv2.imwrite(output_binary_path, binary_img)
    print(f"Binary image for contours saved as {output_binary_path}")
    contours, _ = cv2.findContours(binary_img, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)
    img_contours = img_original.copy()
    cv2.drawContours(img_contours, contours, -1, (0, 255, 0), 2)
    # Save the image with drawn contours
    cv2.imwrite(output_contours_path, img_contours)
    print(f"Image with contours saved as {output_contours_path}")
    print(f"Number of contours found: {len(contours)}")
    # Display both the binary image and the image with contours
    plt.figure(figsize=(15, 7))
    plt.subplot(1, 2, 1)
    plt.imshow(binary_img, cmap='gray')
    plt.title("Binary Image for Contours")
    plt.axis('off')
    plt.subplot(1, 2, 2)
    plt.imshow(cv2.cvtColor(img_contours, cv2.COLOR_BGR2RGB)) # Convert BGR to RGB for
matplotlib
    plt.title("Detected Contours")
```

```
plt.axis('off')
plt.show()
```

```
Number of contours found: 384
```

Binary Image for Contours

Detected Contours